

Quiz

Let d be the array computed after $n - 1$ iterations of the Bellman-Ford algorithm (applied to a directed graph G) and d' the one computed after n iterations. If there exists a negative cycle in G , then $d'[v] < d[v]$ for every vertex v .

True

☒ False

at least on

Spanningtree 2

12 min

Recall that in each iteration, Boruvka's algorithm adds a set of edges to its current forest. In the worst-case, Boruvka's algorithm needs a linear number of such iterations.

$\Theta(n)$

True

☒ False

höchstens $\log_2 n$

The notion of a minimum spanning tree can also be defined for a graph where there are possibly edges with negative weights (we still just sum the weights and look for the minimum possible value).

True/False: Prim's algorithm also computes a minimum spanning tree on graphs where some edges have negative weights.

☒ True

False

Suppose we have stored a graph $G = (V, E)$ using adjacency lists. The runtimes of the algorithms of Prim, Boruvka and Dijkstra are all at most $O((|V| + |E|) \log |V|)$.

☒ True

False

Let $G = (V, E)$ be an undirected, weighted graph with positive weights. Assume that every path in this graph uses at most h edges (i.e., the length with respect to the number of edges, not the weights, is at most h). Given a source vertex v , we can compute single-source shortest paths (now "shortest" with respect to the weights as usual) in time at most $O(h|E|)$.

☒ True

False

h mal Improve bounds von Bellman-Ford

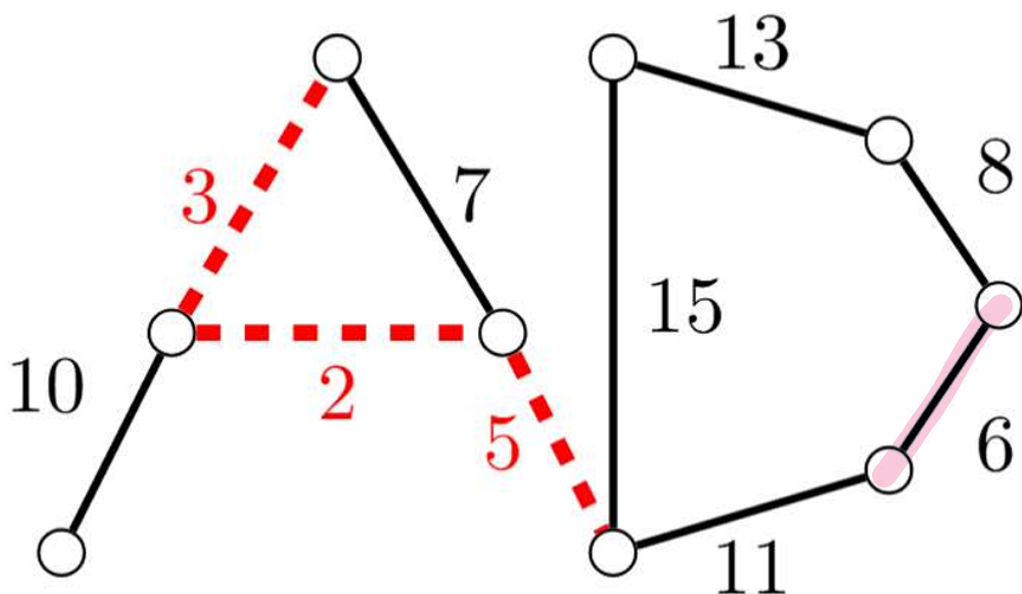
Let G be a weighted undirected graph with distinct edge weights (ie there is no pair of edges with the same weight). Let $e_{\max}$ be the edge of maximum weight. Then, $e_{\max}$ is not in any MST of G .

True

False

0-1-0

We see below a graph on which we have partially executed Kruskal's algorithm. The edges already added by the algorithm are dashed and shown in red. What is the weight of the next edge added?



Answer:

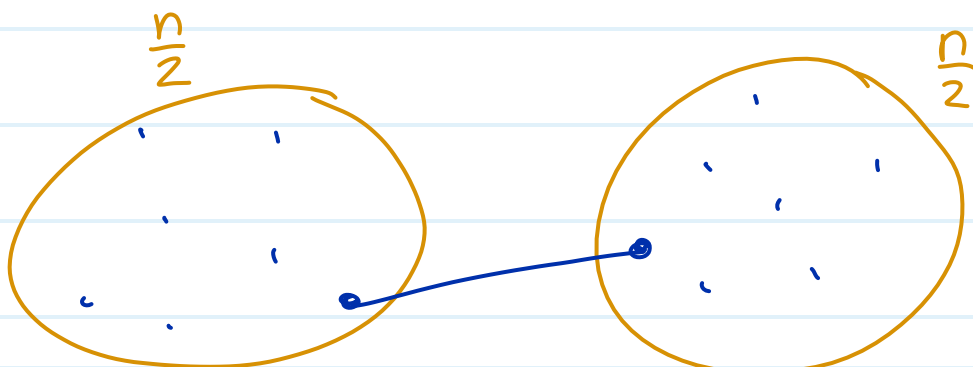
6

For the union-find data structure we saw in lecture, the worst-case runtime of a union step is at most $O(\log n)$.

True

False

can be up to $O(n)$, but the average is $O(\log n)$



As we saw from the lecture, Kruskal's algorithm uses an array `repr` where each vertex stores its representative. Recall that, initially, $\text{repr}[i] = i$ for all i , so there are n different values stored in the array and in the end, only one value will be stored.

True or False: Suppose at some point Kruskal's algorithm has added x edges to the forest. Then, the array `repr` contains exactly $n - x$ distinct values.

True
False

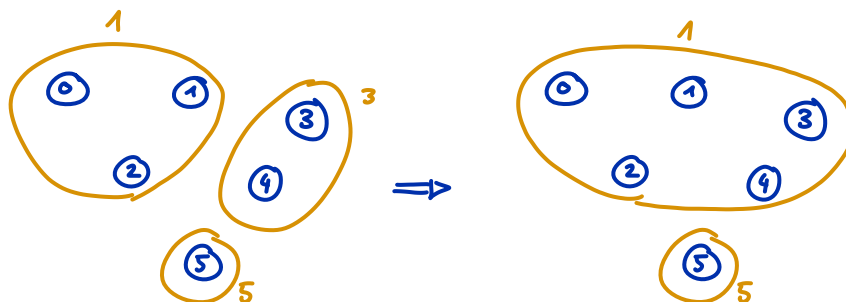
number of different representatives = number of connected components

Consider the following array of representatives in the context of finding an MST of a graph with vertices 0, 1, 2, 3, 4, 5 using a union-find data structure:

`repr` [1, 1, 1, 3, 3, 5].

Now, suppose we unite the components of vertices 1 and 3, using the optimized union procedure which leads to $O(|V| \log |V|)$ runtime for the union steps in Kruskal's algorithm. What is the resulting array?

- a. [1, 1, 1, 3, 3, 5]
- b. [3, 3, 3, 3, 3, 5]
- c. [1, 1, 1, 1, 3, 5]
- ☒ d. [1, 1, 1, 1, 1, 5]



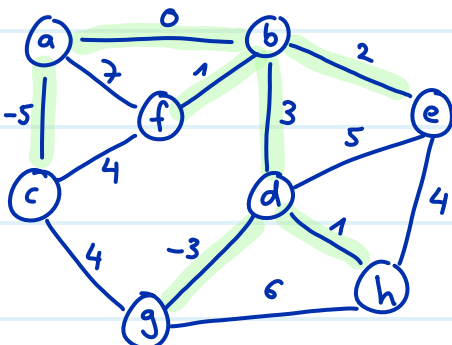
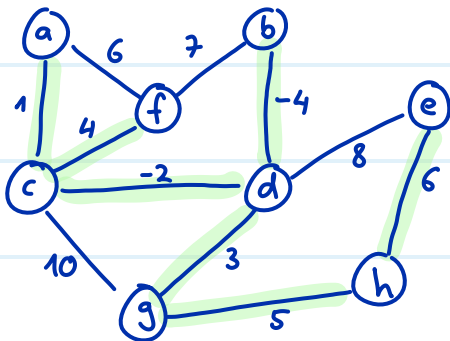
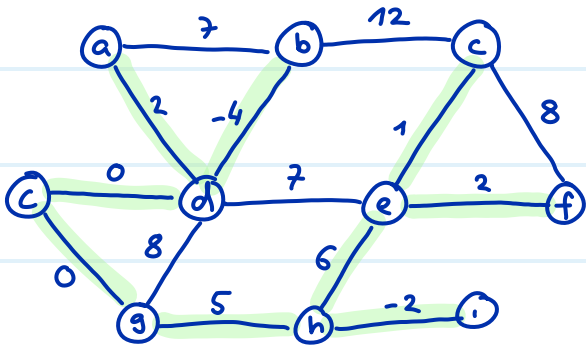
Kruskal

Algorithm 1 KRUSKAL(G)

```

1:  $F \leftarrow \emptyset$ 
2: for  $(u, v) \in E$  aufsteigend sortiert nach Gewicht do
3:   if  $u$  und  $v$  in verschiedenen ZHKs von  $F$  then
4:      $F \leftarrow F \cup \{(u, v)\}$ 
5:   end if
6: end for

```



Algorithm 1 KRUSKAL(G) with Union-Find Logic

Data Structures:

1: $rep[1 \dots n]$ Invariante · $rep[u] = rep[v] \Leftrightarrow ZHK(u) = ZHK(v)$
 2: $members[1 \dots n]$ $members[u]$ ist eine Liste aller Knoten in $ZHK(u)$

3: **Make**(V): $O(n)$

4: **for** $v \in V$ **do**

5: $rep[v] \leftarrow v$

6: $members[rep[v]] \leftarrow \{v\}$

jeder Knoten ist eine eigene ZHK

7: **Same**(u, v): $O(1)$

8: **return** ($rep[u] == rep[v]$)

9: **Union**(u, v): $O(\min\{|ZHK(u)|, |ZHK(v)|\})$

10: **if** $|members[rep[u]]| > |members[rep[v]]|$ **then**

11: Swap u, v

wir wollen möglichst wenige $rep[]$ pointer umhängen

12: **for** x in $members[rep[u]]$ **do**

13: $rep[x] \leftarrow rep[v]$

14: $members[rep[v]] \leftarrow members[rep[v]] \cup \{x\}$

15: **Kruskal**($G = (V, E)$): $O(\underbrace{m \log m}_{\text{sortieren}} + \underbrace{n \log n}_{\text{union find}}) \leq O(m \log m)$ weil $m \geq n-1$

16: $F \leftarrow \emptyset$

17: **MAKE**(V)

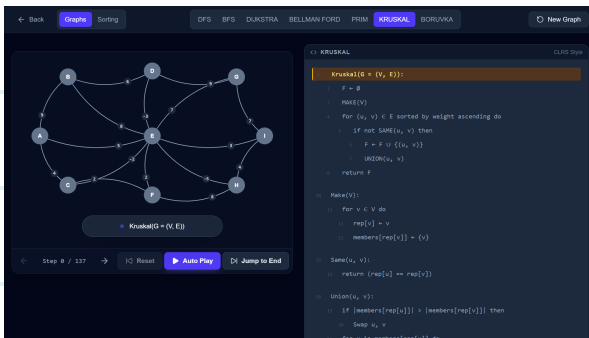
18: **for** $(u, v) \in E$ sorted by weight ascending **do**

19: **if not** **SAME**(u, v) **then**

20: $F \leftarrow F \cup \{(u, v)\}$

21: **UNION**(u, v)

22: **return** F



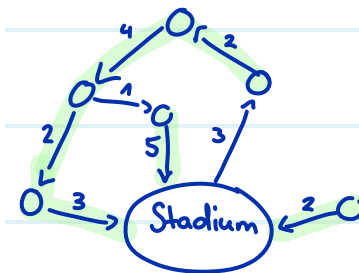
Tricks for Graph Problems

Problem 1a: The One-Way Street Party

The Scenario: You have a map of a city with N houses and exactly **one** large stadium. The city has many one-way streets (a directed graph). Everyone in the city wants to drive from their house to the stadium for a concert.

You need to calculate the shortest path distance from **every** house to the stadium.

The "Obvious" (Inefficient) Approach: Run Dijkstra's algorithm N times, once from every house node. This is computationally expensive ($O(N \cdot E \log V)$).

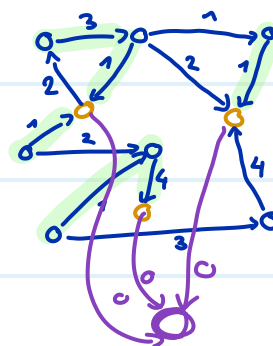


The Creative Construction: Dijkstra's algorithm calculates the distance from *One Source* to *All Destinations*. Here, we have *All Sources* to *One Destination*.

1. **Flip the Graph:** Create a new graph G' where every directed edge $u \rightarrow v$ is replaced by an edge $v \rightarrow u$ with the same weight. (This is called the Transpose Graph).
2. **Algorithm:** Run Dijkstra's algorithm **once** starting from the Stadium in G' .
3. The distance from Stadium to House A in the reversed graph corresponds exactly to the distance from House A to Stadium in the original graph.

Problem 1b: The Mass Evacuation

The Scenario: A storm is approaching. There are N houses in the city (directed graph) and k different storm shelters located at specific nodes. Every household needs to know the distance to the **nearest** shelter. They don't care which one, as long as it is the closest one to their specific house.



The Creative Construction: We have the same "Many-to-One" problem as 1a, but now we also have multiple destinations.

1. **Flip the Graph:** First, reverse all edges so we can calculate paths *originating* from the shelters.
2. **The Super-Source:** Create a virtual node V_{super} .
3. **Connect:** Add a directed edge of **weight 0** from V_{super} to each of the k shelter nodes.
4. **Algorithm:** Run Dijkstra **once** starting from V_{super} .
5. The distance calculated to any specific house node represents the shortest time required to reach *any* of the k shelters.

Problem 2: The Half-Price Coupon

The Scenario: A traveler wants to fly from City A to City B. They have a map of flights with associated costs. They possess a single "50% off" coupon that can be applied to **exactly one** flight edge in their entire journey. Find the path that minimizes the total cost.

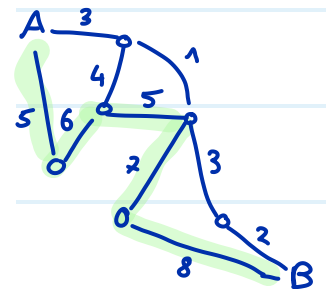
The Creative Construction: The graph must track whether the coupon has been used.

- Layering:** Create two copies of the graph:
 - Layer 0:** Represents the state "Coupon NOT used yet".
 - Layer 1:** Represents the state "Coupon ALREADY used".
- Intra-Layer Edges:** Copy all original edges into both layers with their original weights.
- Inter-Layer Edges (The Jump):** For every flight $u \rightarrow v$ with cost C , draw a directed edge from u (in Layer 0) to v (in Layer 1) with cost $C/2$.
- Algorithm:** Run Dijkstra starting at Node A (Layer 0). The answer is $\text{dist}(B_Layer1)$.

Problem 3: The High-Clearance Truck

The Scenario: You are driving a tall truck from Town A to Town B. The roads have no traffic, but they have old bridges with specific **height limits** (capacities). You want to find a path from A to B such that the **minimum clearance** on that path is maximized (so you can drive the tallest possible truck).

The Challenge: Standard shortest path minimizes the *sum* of weights. Here, we don't care about the sum; we care about the "weakest link" (the lowest bridge).



The Creative Construction: This requires modifying Kruskal

- Sort all edges by height in **descending** order.
- Add edges to the graph one by one (using Union-Find).
- The moment A and B become connected, the weight of the edge that connected them is the answer.

Problem 4: The Most Reliable Path

The Scenario: You are sending a sensitive message across a computer network. Each cable (edge) has a reliability probability P_e (where $0 < P_e \leq 1$). If a message goes through three cables with reliabilities 0.9, 0.9, 0.8, the total success probability is $0.9 \times 0.9 \times 0.8 = 0.648$. Find the path from Start to End with the highest total probability of success.

The Creative Construction: Standard algorithms sum weights ($A + B + C$). Probabilities multiply ($A \times B \times C$).

1. **The Math:** Maximizing the product $P_1 \cdot P_2 \dots$ is equivalent to maximizing $\log(P_1 \cdot P_2 \dots)$, which equals $\sum \log(P_i)$.
2. **The Weights:** Since $P \leq 1$, $\log(P)$ is negative (or zero). Dijkstra requires non-negative weights.
3. **Transformation:** Set every edge weight to $W = -\log(P_e)$.
 - Now, $W \geq 0$.
 - Minimizing the sum of negative logs is mathematically identical to maximizing the product of raw probabilities.
4. **Algorithm:** Run standard Dijkstra with the new weights.

$$\begin{aligned}
 & \max P_1 P_2 \dots P_k \\
 &= \max \log(P_1 P_2 \dots P_k) \\
 &= \max \log(P_1) + \log(P_2) + \dots + \log(P_k) \\
 &= \min -\log(P_1) - \log(P_2) - \dots - \log(P_k)
 \end{aligned}$$

Problem 5: The Currency Arbitrage

The Scenario: You are given a list of exchange rates between currencies (e.g., 1 USD = 0.85 EUR, 1 EUR = 140 JPY). Is it possible to start with 1 unit of currency, trade through a sequence of other currencies, and end up with *more* than 1 unit of the starting currency?

The Creative Construction: This is another multiplicative problem.

1. **Transformation:** Convert exchange rate R to weight $W = -\log(R)$.
2. **The Logic:** If a profit is possible, the product of rates in the cycle is > 1 .
 - $\log(\text{Product}) > 0$
 - Sum of $(-\log(R)) < 0$
3. **Algorithm:** We are looking for a **Negative Cycle**. Run the **Bellman-Ford** algorithm. If it detects a negative cycle, arbitrage is possible.

$$r_1 r_2 \dots r_k > 1$$

$$\Leftrightarrow \log(r_1 r_2 \dots r_k) > 0$$

$$\Leftrightarrow \log(r_1) + \log(r_2) + \dots + \log(r_k) > 0$$

$$\Leftrightarrow -\log(r_1) - \log(r_2) - \dots - \log(r_k) < 0$$

Problem 6a: The Security Clearance

The Scenario: A spy wants to travel from Safehouse A to Safehouse B . The map includes normal roads and one specific high-security bridge connecting intersection U to V . The bridge (U, V) is locked. To cross it, the spy must first acquire a security pass. The pass is located at a specific pickup point, Node P . Find the shortest path from A to B . (Note: The path might not need the bridge, but if it uses the bridge, P must be visited first).

The Creative Construction: You cannot simply run Dijkstra because the edge (U, V) effectively "doesn't exist" until you visit P .

1. **Layering:** Create two copies of the graph:
 - **Layer 0: "No Pass".** This layer contains all edges **except** the bridge (U, V) .
 - **Layer 1: "Has Pass".** This layer contains all edges **including** the bridge (U, V) .
2. **The Transition:** At the pickup node P , add a directed edge from P_{Layer0} to P_{Layer1} with **weight 0**. (This represents instantly picking up the pass).
3. **Algorithm:** Run Dijkstra starting from A_{Layer0} .
 - The answer is the minimum of $\text{dist}(B_{Layer0})$ (reached without ever using the bridge) and $\text{dist}(B_{Layer1})$ (reached after picking up the pass).

Problem 6b: The Spy Car

The Scenario: This extends Problem 6a. The spy still needs to go from A to B and requires a pass from P to cross the bridge (U, V) . However, there is now an additional special node C (Car Garage). If the spy visits C , they acquire a fast car. Driving the car allows them to traverse all subsequent roads in **half the time** (edge weights become $W/2$). Find the fastest path from A to B , considering any combination of picking up the pass and/or the car.

The Creative Construction: We now have two independent state variables: "Has Pass" (Yes/No) and "Has Car" (Yes/No). This requires $2 \times 2 = 4$ layers.

1. **Layering:** Create four copies of the graph:
 - **Layer 00 (No Pass, No Car):** Bridge (U, V) removed. Normal weights W .
 - **Layer 10 (Has Pass, No Car):** Bridge (U, V) exists. Normal weights W .
 - **Layer 01 (No Pass, Has Car):** Bridge (U, V) removed. Weights are $W/2$.
 - **Layer 11 (Has Pass, Has Car):** Bridge (U, V) exists. Weights are $W/2$.

2. Transitions (State Changes):

- At Node P (Get Pass): Add zero-weight edges $P_{00} \rightarrow P_{10}$ and $P_{01} \rightarrow P_{11}$.
- At Node C (Get Car): Add zero-weight edges $C_{00} \rightarrow C_{01}$ and $C_{10} \rightarrow C_{11}$.

3. Algorithm: Run Dijkstra starting from A_{00} .

- The answer is the minimum distance to B across all four layers:
 $\min(\text{dist}(B_{00}), \text{dist}(B_{10}), \text{dist}(B_{01}), \text{dist}(B_{11}))$.

Problem 9: The Feuding Families

The Scenario: You are organizing a wedding dinner. You have a list of N guests. You also have a "List of Enemies": pairs of guests who hate each other and will cause a scene if placed at the same table. You have exactly **two** large tables available. Can you assign every guest to one of the two tables such that no two enemies sit together? If so, how?

The Creative Construction: This is not a shortest path problem. It is a "configuration" problem.

1. **Nodes:** Every guest is a node.
2. **Edges:** Draw an undirected edge between two nodes if they are on the "List of Enemies".
3. **The Goal:** We need to color the nodes with 2 colors (Table 1, Table 2) such that no two connected nodes share a color.
4. **Algorithm:** Run BFS or DFS (Bipartite Check).
 - Pick an unvisited node, assign it Color A.
 - Assign all its neighbors Color B.
 - Assign all *their* neighbors Color A.
 - If you ever find an edge connecting two nodes of the same color, the arrangement is **impossible**.
 - If you finish traversing all components without conflict, the arrangement is **possible**.

Problem 10: The Flat-Rate Commute

The Scenario: You have a city with N subway stations. There are K different subway lines. Each line connects a specific list of stations (e.g., Line 1 connects $\{A, C, F\}$). Entering a line costs a flat ticket fee of \$2.00. Once you have entered a line, you can travel to any other station **on that same line** for free. Switching lines requires exiting and paying the ticket fee for the new line. Find the cheapest cost to get from Start to End.

The "Obvious" (Inefficient) Approach: For every subway line, create a "clique": connect every station on the line to every other station on the line with an edge of weight 2.00. If a line has 100 stations, this creates $\approx 10,000$ edges for just that one line!

The Creative Construction: We can solve this by adding extra "Virtual Nodes" to act as hubs.

1. **Hub Nodes:** Create one virtual node H_k for each subway line k .
2. **Boarding Edges (Pay):** From every Station S that is part of Line k , add a directed edge $S \rightarrow H_k$ with weight 2.00. (This represents paying to enter the line).
3. **Alighting Edges (Free):** From the Hub H_k to every Station S that is part of Line k , add a directed edge $H_k \rightarrow S$ with weight 0. (This represents getting off at your destination).
4. **Algorithm:** Run Dijkstra.
 - Path: $StationA \rightarrow Hub1 \rightarrow StationB$. Total Cost: $2.00 + 0 = 2.00$.
 - Transfer: $StationA \rightarrow Hub1 \rightarrow StationB \rightarrow Hub2 \rightarrow StationC$. Total Cost: $2.00 + 0 + 2.00 + 0 = 4.00$. This construction is linear ($O(N)$) rather than quadratic ($O(N^2)$).

Problem 12: The Electric Vehicle Trip

The Scenario: You are driving an Electric Vehicle (EV) from City A to City B. The car has a battery capacity of C units. Roads connect cities, and driving from u to v consumes $dist(u, v)$ units of charge. You cannot traverse an edge if you don't have enough charge. Some cities have Charging Stations where you can refill your battery to full capacity (C) instantly. Find the shortest travel time to get from A to B.

what if add a charging time of 5 time units?

The Creative Construction: The "state" is not just "where you are", but "how much charge you have".

1. **Nodes:** Create nodes $(u, charge)$ for every city u and every possible integer charge level 0 to C . (Total nodes $N \times (C + 1)$).
2. **Driving Edges:** If there is a road $u \rightarrow v$ with distance D , add edges from $(u, k) \rightarrow (v, k - D)$ for all $k \geq D$. The weight is the travel time.
3. **Charging Edges:** If city u has a charging station, add a directed edge from $(u, k) \rightarrow (u, C)$ for all $k < C$. The weight is the charging time (or 0).